Зміст

1 Програмування 3

1.1 Цілі курсу . . . 3

1.2 Що таке програмування? . . . 3

1.3 Структурне програмування . . . 3

2 Чому Python? Створення та запуск файлів. Робота в інтерпретаторі. 7

2.1 Історія та Глобальний вплив . . . 7

2.1.1 Історія розвитку Python . . . 7

2.1.2 Глобальний вплив та переваги . . . 7

2.2 Переваги мови або чому Python . . . 8

2.2.1 Основні переваги . . . 8

2.2.2 Класифікація мови . . . 9

2.2.3 Недоліки . . . 9

2.3 Середовище для роботи та Інтерпретатор . . . 9

2.3.1 Онлайн середовища розробки . . . 9

2.3.2 Командний рядок інтерпретатора . . . 9

2.3.3 Локальне встановлення Python . . . 10

2.4 Виконання вказівок у Python . . . 10

2.4.1 Інтерактивний режим . . . 10

2.4.2 Виведення повідомлень на екран . . . 11

2.4.3 Програми у файлах . . . 11

2.5 Перевірка коду на помилки в IDLE . . . 11

2.5.1 Запуск програми у IDLE . . . 11

2.5.2 Результати виконання . . . 12

2.5.3 Приклад використання `print` для відлагодження . . . 12

3 Синтаксис Python. Типи Даних 13

3.1 Змінні та Ідентифікатори . . . 13

3.1.1 Правила побудови ідентифікаторів . . . 13

3.2 Зарезервовані слова Python 13
3.2.1 Приклад помилки 14
3.3 Оператори та Вирази 14
3.3.1 Правила форматування коду 15
3.4 Основні Типи Даних і Операції 15
3.5 Приклади. Виділення цифр числа 17
3.5.1 Двоцифрове число 17
3.5.2 Трицифрове число 17
3.6 Вбудовані функції для роботи з числами 18
3.6.1 Функція `abs(a)` 18
3.6.2 Функція `divmod(a, b)` 18
3.6.3 Функція `hex(x)` 18
3.6.4 Функція `oct(x)` 18
3.6.5 Функція `round(x)` 19
3.6.6 Функція `round(x, n)` 19

4 Умовний оператор if 20
4.1 Оператори порівнювання 20
4.2 Синтаксичні форми умовного оператора 21

5 Логічні оператори (and, or, not) 24

6 Генерація випадкових чисел: модуль random 26

7 Оператор циклу `for` та функція `range()` 28

8 Оператор циклу `while` 35

Програмування

1

1.1 Цілі курсу

- ☑ навчитися складати алгоритми
- ☑ навчиться писати фрагменти коду (текст програми)
- ☑ навчиться читати чужий код і використовувати його.

1.2 Що таке програмування?

Слово «програма» походить від грецьких слів «pro» (перед) і «graphein» (писати). Іншими словами, це був акт написання чогось наперед або достроково.

Програмування — це

процес проектування, написання, тестування, вадження і підтримки комп'ютерних програм. У вужчому значенні програмування розглядається як кодування — реалізація у вигляді програми одного чи кількох взаємопов'язаних алгоритмів.

У широкому сенсі процес програмування охоплює і створення, тобто розробку, алгоритмів, і аналіз потреб майбутніх користувачів програмного забезпечення. Технологія програмування постійно розвивається, що призводить до появи нових мов програмування, інструментів розробки і сфер застосування. Тому важливою ознакою програмування є регулярне опанування нових технологій.

Алгоритм — це

скінченна послідовність вказівок на виконання дій, спрямованих на розв'язання певної задачі.

1.3 Структурне програмування

Структурне програмування — це методологія розробки програм, яка ґрунтується на використанні трьох базових конструкцій управління виконанням:

- ☑ слідування (лінійний алгоритм),

- ☑ розгалуження (умовний оператор),
- ☑ цикл (повторення).

Слідування (лінійний алгоритм)

Лінійні алгоритми — це

алгоритми, в яких дії виконуються послідовно одна за одною.

Це означає, що кожна команда лінійного алгоритму виконується строго в заданому порядку. Лінійні алгоритми можуть бути корисними для розв'язання простих задач, таких як знаходження суми двох чисел або переведення одиниць виміру. Наприклад, лінійний алгоритм для знаходження суми двох чисел може мати наступний вигляд:

Приклад: обчислення суми двох чисел.

Алгоритм: Сума двох чисел

Ввести a, b

Обчислити $s = a + b$

Вивести s

Розгалуження (умовний оператор)

Розгалуження — це

алгоритмічна конструкція, яка дозволяє змінювати хід виконання програми залежно від істинності певної умови.

Умовний оператор — це

конкретна реалізація розгалуження у мові програмування. Він перевіряє логічний вираз і, залежно від його значення (true або false), виконує одну з гілок коду.

Приклад: перевірка парності числа.

Алгоритм: Перевірка парності

Ввести n
if $n \bmod 2 = 0$ **then**
  Вивести “Число парне”
else
  Вивести “Число непарне”
end if

Цикл (повторення)

Цикл — це

алгоритмічна конструкція, яка забезпечує багаторазове виконання одних і тих самих дій (команд) доти, доки виконується певна умова або поки не буде досягнуто заданої кількості повторень..

Цикли дозволяють:

- ☑ автоматизувати повторювані обчислення;
- ☑ скоротити код, замінивши багаторазове дублювання;
- ☑ будувати алгоритми для обробки масивів даних, серій чисел, введення/виведення.

Приклад: обчислення суми чисел від 1 до n.

Алгоритм: Сума чисел від 1 до n

Ввести n
$s := 0$
for $i := 1$ **to** n **do**
  $s := s + i$
end for
Вивести s

Теорема структурного програмування

У 1970-х роках Едсгер Дейкстра та Ніклаус Вірт сформулювали принципи структурного програмування.

Вони довели, що будь-який алгоритм можна представити як композицію трьох базових блоків:

- ☑ Слідування — послідовне виконання команд у заданому порядку.
- ☑ Розгалуження — вибір однієї з альтернативних гілок залежно від умови.
- ☑ Цикл — багаторазове виконання команд, доки умова істинна

Логічна повнота конструкцій

Слідування забезпечує лінійність і впорядкованість дій.

Розгалуження додає можливість приймати рішення (умовний вибір).

Цикл дозволяє повторювати дії довільну кількість разів.

Разом ці три конструкції утворюють повний набір керуючих структур, достатній для опису будь-якого алгоритму.

Усі сучасні мови програмування (Python, C++, Java) мають оператори для цих трьох конструкцій (if, for, while).

Навіть складні алгоритми (наприклад, сортування, пошук, обробка даних) можна розкласти на комбінацію цих трьох структур.

Підпрограми (функції, процедури) лише групують ці базові блоки, але не додають нових типів керування.

Висновки

- ☑ Будь-який алгоритм можна побудувати, використовуючи лише три конструкції: слідування, розгалуження та цикл.
- ☑ Володіння цими конструкціями є фундаментом для подальшого вивчення мов програмування.
- ☑ Для математиків важливо розуміти алгоритмічну структуру задач, адже це дозволяє формалізувати процес розв’язання.

Чому Python? Створення та запуск файлів. Робота в інтерпретаторі.

2

2.1 Історія та Глобальний вплив

Створення Python було розпочато Гвідо ван Россумом (Guido van Rossum) у 1991 році, коли він працював над розподіленою ОС Амеба. Йому була потрібна мова, що розширюється, який би забезпечив підтримку системних викликів. За основу були взяті ABC та Модула-3. Як назви він вибрав Python на честь комедійних серій BBC "Літаючий цирк Монті-Пітона а зовсім не за назвою змії. З того часу Python розвивався за підтримки тих організацій, у яких Гвідо працював. Особливо активно мову вдосконалюється нині, коли з неї працює як команда творців, а й ціле співтовариство програмістів з усього світу. І все-таки останнє слово про напрям розвитку мови залишається за Гвідо ван Россумом.

Історія розвитку Python

- ☑ **Python 2.0** — випущений у 2000 році. Серед нововведень: спискові включення, підтримка Unicode, вдосконалений збирач сміття.
- ☑ **Python 3.0** — випущений у 2008 році. Основна особливість — несумісність із попередніми версіями, що вимагало адаптації бібліотек.
- ☑ **Python 2.7** — останній реліз гілки 2.x. Його підтримку офіційно завершено у 2020 році, що ознаменувало перехід до епохи Python 3.

Глобальний вплив та переваги

Python став однією з найпопулярніших мов програмування у світі завдяки простоті синтаксису та універсальності. Його активно використовують такі компанії, як **Google, Netflix, Facebook, Instagram, Spotify** та багато інших.

Основні сфери застосування:

- ☑ **Наука та дослідження** — бібліотеки `NumPy`, `SciPy`, `pandas`.

- **Машинне навчання та штучний інтелект** — `TensorFlow`, `PyTorch`, `scikit-lea`
- **Веб-розробка** — фреймворки `Django`, `Flask`.
- **Автоматизація та DevOps** — скрипти для адміністрування, тестування, CI/CD.
- **Аналіз даних та візуалізація** — `matplotlib`, `seaborn`, `Jupyter Notebooks`.

Таким чином, Python використовується практично всюди — від навчальних проєктів до масштабних систем світових технологічних гігантів.

2.2 Переваги мови або чому Python

Основні переваги

- **Якість програмного забезпечення.** Зручність читання, ясність і висока якість відрізняють Python від інших мов. Єдиноманітність оформлення коду полегшує його розуміння навіть для тих, хто не брав участі у створенні програми.
- **Висока швидкість розробки.** У порівнянні з C, C++ та Java, Python значно підвищує продуктивність праці розробника. Обсяг коду зазвичай становить третину або навіть п'яту частину еквівалентного коду на C++ чи Java. Програми запускаються відразу, минаючи етапи компіляції та зв'язування.
- **Перенесення програмного коду без витрат.** Більшість програм виконується без змін на основних платформах. Перенесення з Linux у Windows часто зводиться до копіювання файлів. Python також підтримує створення графічних інтерфейсів, доступ до баз даних, веб-додатків та інших типів програм.
- **Бібліотеки підтримки.** У складі Python є велика стандартна бібліотека, що охоплює пошук тексту, роботу з мережею та інші функції. Крім того, існують численні сторонні бібліотеки: для веб-розробки, математичних обчислень, ігор тощо. Наприклад, `NumPy` — потужний інструмент для наукових обчислень.
- **Інтеграція компонентів.** Python легко взаємодіє з іншими мовами та платформами: може викликати функції з бібліотек на C/C++, інтегруватися з Java, взаємодіяти з COM та .NET, працювати через протоколи SOAP, XML-RPC, CORBA.
- **Вільне використання.** Python можна використовувати та розповсюджувати абсолютно безкоштовно.

Класифікація мови

1. **Високого рівня.** Код легко читати і писати, він схожий на англійську мову.
2. **Загального призначення.** Python вирішує широкий спектр завдань.
3. **Інтерпретована.** Для виконання коду не потрібна компіляція.

Python часто позиціонують як **об'єктно-орієнтовану мову сценаріїв**, що дозволяє створювати прості програми у вигляді одного файлу, використовуючи об'єкти.

Недоліки

- ☑ **Швидкодія.** Програми на Python можуть працювати повільніше за аналоги, написані компілюючими мовами (наприклад, C).
- ☑ **Байт-код.** Python перетворює вихідний код у байт-код, який інтерпретується. Це забезпечує переносимість програм, але відсутність машинного коду може знижувати продуктивність.

2.3 Середовище для роботи та Інтерпретатор

Python — це не лише мова програмування, а й програмний пакет, що включає **інтерпретатор** — модуль, який створює та виконує інші програми.

Онлайн середовища розробки

Сьогодні є можливість писати код на Python у веб-сервісах, які дозволяють запускати програми прямо у браузері, без встановлення додаткового програмного забезпечення:

- ☑ https://replit.com/~
- ☑ https://ideone.com/
- ☑ https://www.online-python.com/

У цих сервісах достатньо створити акаунт, написати код та запустити його онлайн.

Командний рядок інтерпретатора

Найпростіший спосіб писати й запускати програми на Python — вводити інструкції безпосередньо у **командний рядок інтерпретатора**. Це дозволяє швидко пе-

ревіряти окремі команди та експериментувати з кодом.

Локальне встановлення Python

Щоб працювати локально, потрібно завантажити Python з офіційного сайту:

- ☑ https://www.python.org/downloads/

Після встановлення пакету Python створюється інтерпретатор та бібліотека підтримки. Також доступне інтегроване середовище розробки **IDLE**, яке встановлюється разом із Python і дозволяє писати, запускати та відлагоджувати програми у зручному графічному інтерфейсі.

2.4 Виконання вказівок у Python

Інтерактивний режим

В інтерактивному режимі користувач вводить інструкції та натискає `Enter`, після чого результат відображається рядком нижче.

```
>>> 2 + 2
4
>>> "Hello"
'Hello'
```

Інтерактивний режим дозволяє:

- ☑ швидко перевіряти фрагменти коду;
- ☑ проводити експерименти без створення файлів;
- ☑ одразу виявляти помилки.

Приклад помилки:

```
>>> print(y)
Traceback (most recent call last):
File "<pyshell#15>", line 1, in <module>
print(y)
NameError: name 'y' is not defined
```

Виведення повідомлень на екран

Для виведення інформації використовується команда `print`:

```
>>> print("Hello World")
Hello World
>>> print(2 + 2)
4
```

У інтерактивному режимі можна вводити вирази без `print`, але у файлах інструкцію `print` потрібно писати обов'язково.

Програми у файлах

Програму на Python можна написати у будь-якому текстовому редакторі та зберегти у файл з розширенням `.py`. Наприклад, створимо файл `hello.py`:

```
print("Hello world!")
print("Привіт світ")
input()
```

Запуск програми здійснюється подвійним клацанням по файлу. Такі програми виконуються у консольному вікні («чорному вікні»), де введення здійснюється з клавіатури. Якщо у програмі є помилки, у консолі буде показано повідомлення про них.

Таким чином, Python дозволяє працювати як у інтерактивному режимі, так і створювати повноцінні програми у файлах.

2.5 Перевірка коду на помилки в IDLE

Запуск програми у IDLE

1. Запускаємо середовище **IDLE**.
2. У меню **File** обираємо пункт **Open** та у вікні шукаємо файл `hello.py`.
3. Відкриється нове вікно з текстом програми. У цьому вікні у меню **Run** вибираємо **Run Module** або натискаємо клавішу `F5`.

Результати виконання

- ☑ Якщо у програмі немає помилок, введення та виведення здійснюється у вікні інтерпретатора.
- ☑ Повідомлення про помилки також відображаються у **IDLE**.
- ☑ Якщо програма працює некоректно, але не видає помилок, для пошуку проблем можна вставити додаткові інструкції `print`.

Приклад використання `print` для відлагодження

```
print("Значення змінної x:", x)
```

Це допоможе зрозуміти, які значення мають змінні під час виконання програми.

Синтаксис Python. Типи Даних

3

3.1 Змінні та Ідентифікатори

Змінна - це

ім’я (ідентифікатор), яке може набувати певного значення.

У Python змінні не потрібно оголошувати, значення можна надати одразу за допомогою символу =.

Приклади присвоєння

```
x = 4
s1 = "Hello"
_x = True
x2 = 23.46
summa1 = summa2 = k = 0
```

Ідентифікатор - це

унікальна ознака об’єкта, що дозволяє розрізняти його з іншими.

Правила побудови ідентифікаторів

- ☑ Використовуються лише латинські літери, цифри та знак підкреслення (_).
- ☑ Ідентифікатор повинен починатися з літери або символу підкреслення.
- ☑ Регістр літер має значення (`X` ≠ `x`).
- ☑ Неприпустимі пробіли, точки, спеціальні символи та літери кирилиці.
- ☑ Не можна застосовувати спеціальні символи `$`, `%`, `@`.
- ☑ Заборонено використовувати зарезервовані слова Python (`if`, `for`, `while`, `class`, `def` тощо).

3.2 Зарезервовані слова Python

У Python існує набір **ключових слів**, які не можна використовувати як ідентифікатори (назви змінних, функцій, класів тощо). Ці слова мають спеціальне значення

у синтаксисі мови.

```
False   class     finally  is       return
None    continue  for      lambda   try
True    def       from     nonlocal while
and     del       global   not      with
as      elif      if       or       yield
assert  else      import   pass     break
```

Використання будь-якого з цих слів як назви змінної або функції призведе до помилки.

Приклад помилки

```
>>> class = 5
File "<stdin>", line 1
class = 5
^
SyntaxError: invalid syntax
```

3.3 Оператори та Вирази

Вираз — це

фрагмент мови програмування, що представляє спосіб обчислення деякого значення.

Операнд — це

частина виразу, над яким виконуються операції.

Оператор — це

дія, яка може бути виконана над одним або декількома операндами.

Приклади синтаксису

```
X=4+5 % Присвоєння
Print(x)
S="Hello "+"World"
Print(s)
S="Helo"+\
"World" % Перенесення довгих операторів
```

Правила форматування коду

- ☑ Кожен оператор завершується символом нового рядка.
- ☑ Відступи застосовуються для позначення різних блоків коду (тіла функцій, умовні висловлювання, цикли).
- ☑ Пробіли неприпустимі перед операторами, але необхідні перед вкладеними блоками (`unexpected indent`).
- ☑ Якщо потрібно розмістити кілька операторів в одному рядку, їх розділяють крапкою з комою (`;`).
- ☑ Символ # означає коментар.
- ☑ Багаторядковий коментар укладають у три подвійні лапки в `py` файлі.

3.4 Основні Типи Даних і Операції

Вбудовані типи даних:

- ☑ Числа: `1234`, `3.14`, `3+4j`.
- ☑ Рядки: `'spam'`, `"guido's"`.
- ☑ Списки: `[1, [2, 'three'], 4]`.
- ☑ Словники: `{'food': 'spam', 'taste': 'yum'}`.
- ☑ Кортежі: `(1, 'span', 4, 'U')`.
- ☑ Файли: myfile = open(’eggs’, ‘r’)
- ☑ Множини: set(’abc’). ’a’,’b’, ’c’

Вбудовані числові типи

- ☑ цілі числа (числа без дробової частини), (3, 3465, 45, 42, -4) (з необмеженою точністю)
- ☑ дійсні числа (грубо кажучи, числа з десятковою точкою з плаваючою точкою) (3,15 5,68)

- ☑ 8-річні, починаються нуль про 0o77 = 63 0o10 = 8
- ☑ 16-річні, починаються нуль x 0xFF=255 0xff=255 • - двійкові, починаються нуль b 0b1111 = 15 0b10 = 2
- ☑ - комплексні числа з уявною частиною, (3+4j), (3+4J)(з допомогою великої та маленької) літер)

Символ множення перед j не вказується.

```
>>>z1=2+3j
>>>z2=4+8J
>>>print(z1, z2)
(2+3j) (4+8j)
```

Комплексні числа можуть бути створені за допомогою функції complex(реч _ частина, уявна _ частина)

```
>>>z3=complex(34,2)
>>>print(z3)
(34+2j)
```

Операції з числовими типами

Оператор	Опис	Приклад	Результат
+	Додавання	5+7	12
-	Віднімання	55-10	45
*	Множення	4*6	24
/	Ділення	7 / 2	3.5
//	Цілочисельне ділення	7 // 2	3
%	Остача від ділення	7 % 3	1
**	Піднесення до степеня	3 ** 4	81

Результатом операції буде новий об’єкт.

2 ● ● 100#2100

1267650600228229401496703205376

Зверніть увагу на результат останньої операції: у Python 3 цілі числа автоматично забезпечують **необмежену** точність. Наприклад, ви можете спробувати обчислити 2 ступенем 1 000 000 (але навряд чи варто це робити, оскільки на екран буде виведено число довжиною понад 300 000 знаків, що може зайняти тривалий

час!).

> len(str(2 ** 1000000))

Скільки цифр у дійсно великому числі

301030

3.5 Приклади. Виділення цифр числа

Двоцифрове число

Нехай маємо число `n = 47`. Щоб виділити окремі цифри:

Двоцифрове число

```
n = 47
d1 = n // 10    # перша цифра (десятки)
d2 = n % 10     # друга цифра (одиниці)
print("Десятки =", d1, "Одиниці =", d2)
```

Результат:

```
Десятки = 4 Одиниці = 7
```

Трицифрове число

Нехай маємо число `m = 583`. Щоб виділити окремі цифри:

Трицифрове число

```
m = 583
sot = m // 100          # сотні
des = (m // 10) % 10    # десятки
od = m % 10             # одиниці
print("Сотні =", sot, "Десятки =", des, "Одиниці =", od)
```

Результат:

```
Сотні = 5 Десятки = 8 Одиниці = 3
```

Пріоритет операторів - порядок виконання тут

3.6 Вбудовані функції для роботи з числами

Python надає низку **вбудованих функцій**, які можна застосовувати до числових типів.

Функція `abs(a)`

Повертає модуль числа.

Приклад

```
x = -12
print(abs(x))    # 12
```

Функція `divmod(a, b)`

Виконує ділення `a` на `b` та повертає пару чисел: частку і залишок.

Приклад

```
x = 23
y = 7
print(divmod(x, y))    # (3, 2)
```

Функція `hex(x)`

Конвертує ціле число у рядок із шістнадцяткових цифр.

Приклад

```
x = 15
print(hex(x))    # '0xf'
```

Функція `oct(x)`

Конвертує ціле число у рядок із вісімкових цифр.

Приклад

```
x = 15
print(oct(x))    # '0o17'
```

Функція `round(x)`

Округлює число до найближчого цілого.

Приклад

```
print(round(3.0254))    # 3
```

Функція `round(x, n)`

Округлює число `x` з точністю до `n` знаків після коми.

Приклад

```
print(round(3.0254, 2))    # 3.03
```

Умовний оператор if

4

За замовчуванням хід виконання програми є лінійним: інструкції виконуються послідовно — рядок за рядком, від першого до останнього. Послідовне виконання є базою, але реальні алгоритми вимагають гнучкості та адаптивності.

У більшості задач програма повинна приймати рішення залежно від вхідних даних чи поточного стану обчислень. Для цього використовуються розгалуження (або умовні переходи).

Оператор if в Python — це

умовний оператор, який дозволяє програмі приймати рішення та виконувати певний блок коду лише тоді, коли виконується задана умова

В основі прийняття рішень у програмі лежать оператори порівняння (відношення). Вони дозволяють зіставляти значення виразів і повертають результатом булеве значення — **True (істина)** або **False (хибність)**.

4.1 Оператори порівнювання

Python підтримує базові математичні операції відношення:

```
Дорівнює:                a == b
Не дорівнює:             a != b
Менше ніж:               a < b
Менше або дорівнює:     a <= b
Більше ніж:              a > b
Більше або дорівнює:    a >= b
```

Зверніть увагу:

Оператор перевірки на **рівність** позначено двома знаками ==, оскільки один знак = в Python зарезервовано для операції **присвоєння**.

На відміну від багатьох інших мов програмування, Python дозволяє використовувати ланцюгові порівняння (подвійні нерівності), що звично для математиків, наприклад: **a < x <= b** замість запису через логічне «і».

Отримані булеві вирази є фундаментом для керування потоком виконання про-

грами: вони використовуються в умовних конструкціях (if, elif, else) та циклах (while).

4.2 Синтаксичні форми умовного оператора

Залежно від кількості напрямків (гілок) виконання програми, в Python застосовують три основні форми розгалуження:

1. Неповна форма (коротке розгалуження)

Використовується, коли певну дію потрібно виконати **тільки** за умови, що вираз-умова є істинним (`True`). Якщо умова хибна (`False`), блок коду просто пропускається.

```
Неповна форма
if вираз_умова1:
    оператор1
    оператор2
```

2. Повна форма (двохгілкове розгалуження)

Застосовується, коли є два альтернативних шляхи: один виконується при істинності умови (`True`), а інший (блок `else`) — якщо умова хибна (`False`).

```
Повна форма
if вираз_умова1:
    оператор1
    оператор2
else:
    Блок_операторів_3
```

3. Багатогілкова форма (багатоальтернативне розгалуження)

Використовується для перевірки серії послідовних умов за допомогою конструкцій `elif` (скорочення від *else if*). Умови перевіряються зверху вниз: виконується блок першої умови, яка виявиться істинною, після чого весь оператор `if` завершує роботу.

```
Багатогілкова форма
if вираз_умова1:
    Блок_операторів_1
elif вираз_умова2:
   Блок_операторів_2
elif вираз_умова_n:
    Блок_операторів_n
else:
    Блок_операторів_n+1
```

Важливі синтаксичні особливості Python:

☑ **Двокрапка (:):** є обов'язковою в кінці кожного рядка з `if`, `elif` та `else`. Вона позначає початок відповідного блоку коду.

Приклад 1. Користувач вводить число та отримує відповідь яке воно позитивне негативне або нуль.

```
x=int(input("Введіть x "))
if x>0:
    print("x позитивне")
else:
    print("x негативне або нуль")
input()
```

Приклад 2. Користувач вводить два числа та отримує відповідь яке більше або дорівнюють.

```
A=input()
B=input()
if A>B: print("A більше") #можно писати у тому ж рядку один оп-р
elif A==B: print("A дорівнює B")
else: print("A менше B")
```

Приклад 3. Користувач отримує відповідь чи оплачений чек (ціна записана у коді, або можна ввести з клавіатури). (звернути увагу на імена змінних

```
print ("Привіт")
price1 = 50 #price1=int(input())
price2 = 32 #price2=int(input())
if price1+ price2 > 99 :
    print ("Сума не достатня")
else:
    print ("Чек оплачений")
print ("До побачення")
```

Приклад 4. Вивести на екран місяці відповідної попи року

```
s=input("Введіть пору року")
if s=="зима":
    print("грудень")
    print("січень")
    print("лютий")
elif s=="весна":
    print("березень")
    print("квітень")
    print("травень")
elif s=="літо":
    print("червень")
    print("липень")
    print("серпень")
else:
    print("вересень")
    print("жовтень ")
    print("листопад")
print ("До побачення")
```

Логічні оператори (and, or, not)

5

Прості умовні вирази (операції відношення) можна об'єднувати у складні **складені умови** за допомогою **логічних операторів** (булевих зв'язок). В Python використовуються три базові логічні оператори: **`and`, `or` та `not`**.

1. Кон'юнкція (`and` — логічне «І»)

Оператор **`and`** повертає **`True`** тоді й лише тоді, коли **обидва** операнди є істинними. Якщо хоча б один із виразів хибний (**`False`**), увесь складений вираз набуває значення **`False`**.

Пріоритет оператора **`and`** нижчий за пріоритет операторів порівняння (>, <, == тощо), тому у виразах на зразок:

Перевірка діапазону

```
if length > 8 and length < 20:
    print("Довжина відповідає вимогам")
```

дужки навколо окремих порівнянь писати не обов'язково.

2. Диз'юнкція (`or` — логічне «АБО»)

Оператор **`or`** повертає **`True`**, якщо **хоча б один** із операндів є істинним (або обидва одночасно). Вираз набуває значення **`False`** лише тоді, коли всі складові частини є хибними.

Приклад диз'юнкції

```
if age < 18 or age >= 60:
    print("Пільгова категорія")
```

3. Заперечення (`not` — логічне «НЕ»)

Унарний оператор **`not`** інвертує (змінює на протилежне) значення булевого виразу: **`True`** перетворюється на **`False`**, а **`False`** — на **`True`**.

Приклад заперечення

```
if not is_valid:
    print("Помилка введення даних")
```

Таблиця 5.1: Таблиця істинності для логічних операцій

Значення операндів		Результат операції		
X	Y	not X	X and Y	X or Y
False	False	True	False	False
False	True	True	False	True
True	False	False	False	True
True	True	False	True	True

Особливості Python:

- ☑ **Пріоритет виконання:** Найвищий пріоритет має `not`, далі йде `and`, і найнижчий — `or`. Для зміни порядку обчислення використовують круглі дужки `()`.
- ☑ **Ліниві обчислення (Short-circuit evaluation):** Python обчислює логічні вирази зліва направо і зупиняється, як тільки результат стає однозначним (наприклад, якщо у виразі з `and` перший операнд `False`, другий операнд навіть не обчислюється).

Генерація випадкових чисел: модуль random

6

При розв'язанні багатьох математичних задач, моделюванні ймовірнісних процесів (метод Монте-Карло) або створенні ігор виникає потреба у використанні **випадкових чисел**. У Python для цього призначений вбудований модуль `random`.

Щоб отримати доступ до функцій модуля, його необхідно імпортувати на початку програми:

Імпорт модуля

```
import random
```

Основні функції модуля random

1. Дійсні числа: `random.random()` та `random.uniform()`

- ☑ `random.random()` — повертає випадкове дійсне число x у напіввідкритому інтервалі $[0.0, 1.0)$.
- ☑ `random.uniform(a, b)` — повертає випадкове дійсне число x у відрізку $[a, b]$.

Генерація дійсних чисел

```
import random
# Випадкове число від 0.0 до 1.0
p = random.random()
print("Імовірність",p)

# Випадкова температура в межах [-5.0, 15.0]
temp = random.uniform(-5.0, 15.0)
print("Температура:", temp)
```

2. Цілі числа: `random.randint()` та `random.randrange()`

- ☑ `random.randint(a, b)` — повертає випадкове ціле число $n \in [a, b]$ (включаючи обидва кінці a та b).

- ☑ `random.randrange(start, stop, step)` — повертає випадковий елемент із послідовності `range(start, stop, step)`.

Генерація цілих чисел

```
import random
# Моделювання кидка грального кубика (від 1 до 6)
dice = random.randint(1, 6)
print("Випало число:", dice)

# Випадкове парне число від 10 до 98
even_num = random.randrange(10, 100, 2)
print("Парне число:",even_num)
```

Відтворюваність результатів: `random.seed()`

Оскільки Python генерує **псевдовипадкові числа** за допомогою алгоритму (Вихр Мерсенна), послідовність обчислень залежить від початкового значення (значення ініціалізації — *seed*).

Функція `random.seed(a)` фіксує початковий стан генератора. Це є критично важливим у наукових та математичних обчисленнях для **відтворюваності результатів експерименту**.

Фіксація генератора

```
import random

random.seed(42)  # Задаємо фіксоване зерно
print(random.randint(1, 100))  # Завжди повертатиме один і той самий р
```

Оператор циклу `for` та функція `range()`

7

Оператори циклів використовують у випадках, коли в програмі необхідно багаторазово виконати певну послідовність команд. У мові Python основним циклом із заздалегідь відомою кількістю повторень є цикл `for`.

```
Синтаксис циклу for
for параметр in range(start, stop, step):
    блок_операторів
```

1. Генератор послідовностей `range()`

Найчастіше для формування послідовностей чисел у циклі `for` використовується вбудована функція `range()`. Вона може приймати від 1 до 3 аргументів:

- ☑ `range(stop)` — генерує числа від 0 до $stop - 1$ із кроком 1.
- ☑ `range(start, stop)` — генерує числа від $start$ до $stop - 1$ із кроком 1.
- ☑ `range(start, stop, step)` — генерує числа від $start$ до $stop - 1$ із кроком $step$.

Виклик `range()`	Згенерована послідовність	Примітка
`range(5)`	`0, 1, 2, 3, 4`	Від 0 до 4
`range(1, 6)`	`1, 2, 3, 4, 5`	Від 1 до 5
`range(2, 10, 2)`	`2, 4, 6, 8`	Парні числа від 2 до 8
`range(10, 0, -1)`	`10, 9, 8, 7, 6, 5, 4, 3, 2, 1`	Зворотний відлік

Таблиця 7.1: Приклади роботи функції `range()`

Якщо передати в range() лише одне число, за замовчуванням відлік починається з 0, а верхня межа не включається

```
for i in range(5):
    print(i)
```

Результат виведення : 0, 1, 2, 3, 4

range(start, stop) починає з числа start (включно) і зупиняється перед stop (не включаючи його).

```
for i in range(1,5):
    print(i)
```

Результат виведення: 1, 2, 3, 4

Третій параметр — це крок (step). range(1, 10, 2) починає з 1 і додає 2 на кожному кроці, генеруючи непарні числа до 10

```
for i in range(1,10,2):
    print(i)
```

Результат виведення: 1, 3, 5, 7, 9

Якщо крок від'ємний (-1), значення будуть зменшуватися. Цикл починається з 10 і зупиняється перед 0.

```
^^I
for i in range(10,0,-1):
    print(i)
```

Результат виведення: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

2. Базові алгоритми з циклом `for`

Приклад 1. Обчислення суми натуральних чисел від 1 до n

```
n = int(input("Введіть n "))
s = 0
for i in range(1, n + 1):
    s += i

print(f"Сума чисел від 1 до {n} дорівнює: {s}")
```

input(...) запитує у користувача число та приймає його як рядок (текст).

int(...) перетворює введений текст у ціле число та зберігає його в змінну n.

$s = 0$ створюється змінна-накопичувач s із початковим значенням 0. У ній буде зберігатися підсумкова сума.

for i in range(1, n + 1): цикл for перебирає числа по порядку.

range(1, n + 1) генерує послідовність від 1 до n включно (верхня межа $n + 1$ не

включається в діапазон, тому цикл зупиняється саме на n).

s += i на кожній ітерації циклу поточне число i додається до суми s (це скорочений запис для $s = s + i$).

print(f"Сума чисел від 1 до n дорівнює: s")

За допомогою f-рядка на екран виводиться підсумковий результат.

Приклад 2. Обчислення факторіала n!

```
n = int(input("Введіть n "))
fact = 1

for i in range(1, n + 1):
    fact *= i

print(f"{n}! = {fact}")
```

Приклад 3. Табулювання функції y = sin(x) на відрізку [a, b]

```
import math

a = float(input("Введіть початок відрізка a "))
b = float(input("Введіть кінець відрізка b "))
n = 10
h = (b - a) / n

print(" x | sin(x)")
print("-------------")
for i in range(n + 1):
    x = a + i * h
    y = math.sin(x)
    print(f"{x:6.2f} | {y:6.4f}")
```

Форматоване виведення (виділений рядок):

$\{x : 6.2f\}$ — виводить значення x: занімає 6 символів у ширину та округлюється до 2 знаків після коми.| — виводить розділювальну риску між стовпчиками.

$\{y : 6.4f\}$ — виводить значення y ($\sin(x)$): шириною в 6 символів та округлене до 4 знаків після коми.

3. Аналіз дільників числа та перевірка на простоту

Математична довідка

Просте число — це натуральне число, більше за 1, яке має точно два різні натуральні дільники: 1 та саме це число. Якщо число n має дільник, відмінний від 1 та n, то найменший такий дільник не перевищує $\lfloor\sqrt{n}\rfloor$.

Приклад 4. Виведення всіх дільників числа

```
n = int(input("Введіть натуральне число n "))
count=0
for i in range(1, n + 1):
    if n % i == 0:
        print(i)
    count=count+1
print(f"Кількість дільників: {count}")
```

Пошук дільників: Програма перебирає всі цілі числа i від 1 до n включно і перевіряє умову n

Виведення та лічильник: Якщо число i є дільником:воно одразу виводиться на екран (print(i)); лічильник count збільшується на 1. Наприкінці програма виводить загальну кількість знайдених дільників.

4. Обчислення нескінченних математичних рядів

4.1. Сума геометричного ряду

Обчислимо суму $S = \sum_{n=0}^{N} x^n = 1 + x + x^2 + \cdots + x^N$ ($|x| < 1$).

Приклад 5. Сума геометричного ряду

```
x = float(input("Введіть x (|x| < 1) "))
n = int(input("Введіть кількість членів N "))

s = 0.0
term = 1.0

for i in range(n):
    s += term
    term *= x

print(f"Сума ряду: {s:.6f}")
print(f"Точна сума 1/(1-x): {1 / (1 - x):.6f}")
```

4.2. Обчислення експоненти e^x через ряд Тейлора

Приклад 6. Обчислення e^x

```
import math

x = float(input("Введіть x "))
n = int(input("Кількість членів ряду "))

s = 0.0
term = 1.0

for i in range(1, n + 1):
    s += term
    term *= x / i

print(f"Наближене значення e^{x} = {s:.8f}")
print(f"Точне значення (math.exp): {math.exp(x):.8f}")
```

f"..."(f-рядок) — дозволяє вставляти значення змінних безпосередньо всередину тексту за допомогою фігурних дужок .

$\{s : .8f\}$ — бере значення змінної s (суму ряду) та форматує його як число з плаваючою крапкою, залишаючи рівно 8 знаків після коми.

Цей рядок показує наближене значення експоненти e^x , вираховане через суму перших n членів ряду Тейлора.

5. Управління циклом: `break`, `continue` та `else`

- ☑ `break` — достроково припиняє виконання циклу.
- ☑ `continue` — пропускає залишок тіла циклу та переходить до наступної ітерації.
- ☑ `else` — блок коду, який виконується після завершення циклу, якщо вихід з нього **не** відбувся за допомогою `break`.

Приклад 7. Пошук першого від'ємного числа з використанням break та else

```
for i in range(5):
    val = int(input("Введіть число "))
    if val < 0:
        print(f"Знайдено перше від'ємне число: {val}")
        break
else:
    print("Усі введені числа є невід'ємними.")
```

6. Оператори інкременту та особливості `float`

У Python відсутні оператори `++` та `--`. Замість них використовуються комбіновані оператори присвоєння:

- ☑ `x += 1` замість `x = x + 1`
- ☑ `x -= 1` замість `x = x - 1`
- ☑ `x *= 2` замість `x = x * 2`

Обережно з дійсними числами (float)!

Числа з плаваючою крапкою зберігаються в пам'яті комп'ютера у двійковій системі з обмеженою точністю (53 біти). Через це обчислення можуть містити незначну похибку:

Особливості float

```
>>> 0.1 + 0.1 + 0.1
0.30000000000000004
```

Для порівняння дійсних чисел завжди використовуйте функцію `math.isclose(a, b)` або порівняння з точністю ε: `abs(a - b) < 1e-9`.

Оператор циклу `while`

8

Оператори циклів використовують у випадках, коли в програмі необхідно багаторазово виконати певну послідовність команд. У мові Python основним циклом із умовою повторення (циклом із передумовою) є цикл `while`.

```
Синтаксис циклу while
while вираз_умови:
    блок_операторів
```

Блок операторів циклу `while` повторюється до того часу, поки результатом логічного висловлювання умови повтору є значення `True`. Вираз умови обчислюється перед кожним повтором циклу, і вихід з циклу (тобто перехід до оператора, наступного за оператором `while`) здійснюється в тому випадку, якщо вираз умови має значення `False`.

Приклад 1. Виведення значень лічильника від 0 до 4

```
i = 0
while i < 5:
    print("значення лічильника =", i)
    i += 1  # i = i + 1

print("end")
```

Покроковий аналіз виконання:

Задається початкове значення змінної $i = 0$.

Перевіряється умова $i < 5$. Оскільки вона виконується (`True`), відбувається перехід до операторів всередині циклу (виводимо значення i на екран та збільшуємо на 1, тобто $i = 0 + 1 = 1$).

Знову повертаємось до перевірки умови на початку $i < 5$. Вона виконується $\rightarrow$ виконуються оператори виводу на екран та збільшення i, і так до тих пір, поки значення i не стане дорівнювати 5. Тоді оператори всередині циклу вже не будуть виконуватись, виконання перейде до оператора `print("end")`.

1. Керування циклом: `break`, `continue`, `pass` та блок `else`

```
Структура керування циклом while
while вираз_умови_test:
    блок_операторів_1
    if умова_1:
        break     # вийти з циклу, пропустивши else
    if умова_2:
        continue  # перейти на початок циклу до виразу_умови_test
    блок_операторів_2
else:
    оператори_else # виконуються, якщо вираз_умови став False (без bre
```

- ☑ **`break`** — вихід з циклу із пропуском блоку `else`.
- ☑ **`continue`** — перехід на початок циклу до перевірки умови.
- ☑ **`pass`** — порожня інструкція. Не виконує жодних дій і використовується у випадках, коли синтаксис мови потребує наявності інструкції, але поки що жодних корисних дій виконати не потрібно:

```
if True:
    pass  # інструкції будуть написані пізніше

while True:
    pass  # порожній нескінченний цикл
```

2. Типові застосування циклу `while`

Приклад 2. Інтерактивна перевірка умов (перевірка стану)

```
x = input("Двері відчинені? ")
while x.lower() != "так":
    print("стукаємо")
    x = input("Двері відчинені? ")
print("Заходимо")
```

Примітка: Блок операторів у `while` може не виконатись жодного разу, якщо вираз-умова від самого початку має значення `False`.

Приклад 3. Табулювання функції y = sin(x) до першого від'ємного значення

```
import math

x = 0.0
while math.sin(x) >= 0:
    print(f"sin({x:.1f}) = {math.sin(x):.4f}")
    x += 0.1
```

Приклад 4. Запит на введення додатнього числа з валідацією

```
x = int(input("Введіть додатнє x = "))
while x <= 0:
    x = int(input("Введіть додатнє x = "))
print("Дякую")
```

Програма буде виводити на екран повідомлення та перевіряти, яке число ввід користувач. Якщо $x <= 0$, то повідомлення з запрошенням вводу буде з'являтися знов та знов. Якщо додатнє, то цикл завершиться, бо умова $x <= 0$ перестане виконуватись, та виконаэться рядок коду, що йде одрвзу після циклу while.

Приклад 5. Обчислення суми введених чисел до введення 0 (стоп-сигнал)

```
s = 0.0
x = float(input())
while x != 0:
    s += x
    x = float(input())
print("Сума =", s)
```

Ініціалізується змінна s, яка буде накопичувати суму введених чисел.

Користувач вводить число з клавіатури, яке перетворюється у тип float (дійсне число).

Запускається цикл, який працює доти, доки введене число не дорівнює нулю.

До змінної s додається введене число x.

Користувач вводить наступне число. Якщо це знову не нуль — цикл повторюється.

Коли користувач вводить 0, цикл завершується, і програма виводить загальну суму всіх введених чисел.

Приклад 6. Обчислення суми нескінченного ряду (розклад exp(x)) з точністю eps

```
# Розклад e^x = 1 + x + x^2/2! + ... + x^n/n! + ...
import math
# Початкові дані та параметри
x = 1.0        # Значення x, для якого обчислюємо e^x
eps = 1e-9     # Точність обчислень (10^-9)
s = 0.0        # Змінна для накопичення суми ряду
term = 1.0     # Перший доданок ряду (при n = 0, x^0 / 0! = 1)
i = 1          # Лічильник ітерацій та порядковий номер
#наступного додатка

# Цикл виконується, поки модуль поточного доданка більший за
#задану точність eps
while abs(term) > eps:
    s += term           # Додаємо поточний доданок до  суми
    term = term * x / i # Обчислюємо наступний доданок
    i += 1              # Збільшуємо номер ітерації на 1
print(f"Сума s = {s}")
print(f"Ітерацій: {i - 1}")
```

Примітка: Будь-який цикл `for` можна записати за допомогою `while`, але зворотне твердження взагалі кажучи невірно, оскільки `while` призначений для циклів з невідомою заздалегідь кількістю повторень.

3. Проблема «зациклювання»

Іноді при неуважному використанні оператора `while` виникає так зване **зациклювання** — нескінченне повторення тіла циклу. Це може статися з двох основних причин:

1. Змінні, що є частиною висловлювання умови циклу, не змінюють усередині циклу своїх значень або приймають некоректні значення, у результаті чого вираз умови завжди має значення `True`.
2. Некоректним є сам вираз умови циклу (наприклад, порівняння дійсних чисел типу `float` на строгу рівність).

Приклад зациклювання 1: Нескінченний цикл при перериванні Ctrl+C

```
while True:
    print("Натисніть Ctrl+C для зупинки")
```

while True: — створює нескінченний цикл. Оскільки умова True (істина) завжди залишається True і ніколи не змінюється на False, цикл буде виконуватися вічно.

print(”Натисніть Ctrl+C для зупинки”) — тіло циклу, яке безперервно виводить це повідомлення в консоль/термінал.

Зациклювання виникає тому, що у коді немає жодної умови виходу (наприклад, оператора break або зміни логічного виразу), тому програма самостійно зупинитися не може.

Щоб примусово перервати виконання нескінченного циклу в терміналі чи консолі, використовують комбінацію клавіш Ctrl + C. Вона надсилає програмі сигнал переривання (KeyboardInterrupt), що викликає зупинку виконання коду.

Приклад зациклювання 2: Небезпечне порівняння дійсних чисел (float)

```
# Некоректний код
x = 0.0
while x != 1.0:# умова не перестане виконуватись ніколи
    print(f"x = {x:.1f}")
    x += 0.3

# Правильний варіант:
x = 0.0
while x < 1.0:
    print(f"x = {x:.1f}")
    x += 0.3
```

4. Складніші приклади та алгоритми

Приклад 7. Середнє арифметичне простих чисел до введення 0

```
x = int(input())  # Зчитуємо перше ціле число з клавіатури
k = 0             # Лічильник знайдених простих чисел
S = 0             # Змінна для збереження суми простих чисел

# Цикл триває, поки користувач не введе 0 (ознака кінця введення)
while x != 0:
    is_prime = True  # Прапорець: припускаємо, що число x є простим

    if x <= 1:
        is_prime = False  # Числа <= 1 не є простими
    else:
        # Перевіряємо наявність дільників від 2 до  кореня з x включно
        for i in range(2, int(x**0.5) + 1):
            if x % i == 0: # Якщо x ділиться на i без остачі
                is_prime = False  # то воно не просте, і далі
                break             # можна не перевіряти

    # Якщо число виявилося простим
    if is_prime:
        k += 1  # Збільшуємо кількість простих чисел на 1
        S += x  # Додаємо число до загальної суми

    x = int(input())  # Зчитуємо наступне число для наступної ітерації

# Підбиття підсумків після виходу з циклу
if k == 0:
    print("Простих чисел немає")  # Якщо жодного простого числа не бул
else:
    # Обчислюємо та виводимо середнє арифметичне (сума / кількість)
    print("Середнє арифметичне простих:", S / k)
```

Програма почергово приймає цілі числа, поки не буде введено 0 (сигнал зупинки). Для кожного введеного числа перевіряється, чи є воно простим (тобто більшим за 1 і ділиться лише на 1 та саму себе). Якщо число виявилося простим, програма збільшує лічильник k на 1 та додає це число до загальної суми S. Після введення 0 програма ділить суму простих чисел на їхню кількість (S / k) або повідомляє, що простих чисел не було введено.

Приклад 8. Перевірка наявності цифри 4 у натуральному числі

```
N = int(input('N = '))  # Зчитуємо натуральне число
temp = N                # Створюємо тимчасову копію числа,
# щоб не змінювати оригінальне N

# Цикл триває, поки у тимчасовій змінній залишаються цифри
#(число більше 0)
while temp > 0:
    x = temp % 10       # Отримуємо останню цифру числа
                        #(остача від ділення на 10)

    if x == 4:          # Перевіряємо, чи дорівнює ця цифра 4
        print('Є 4') # Виводимо повідомлення, що цифру 4 знайдено
        break   # Достроково перериваємо цикл,
                # оскільки перевіряти далі немає потреби

    temp //= 10         # Відкидаємо останню цифру (цілочисельне ділен

else:
   # Блок else виконується лише тоді, коли цикл while завершився приро
   # (тобто temp стало <= 0), і не був перерваний оператором break
   print('Немає 4')
```

Користувач вводить число N. Програма створює його копію temp, щоб зберегти оригінальне значення. Отримує останню цифру числа за допомогою temp % 10. Якщо цифра дорівнює 4, виводиться ’Є 4’, і цикл негайно зупиняється (break). Якщо цифра не 4, вона відкидається за допомогою цілочисельного ділення temp на 10. Якщо цикл закінчився і цифру 4 так і не знайшли, спрацьовує блок else, який виводить ’Немає 4’.

Приклад 9. Алгоритм Евкліда пошуку найбільшого спільного дільника (НСД)

```
a = int(input("a = "))
b = int(input("b = "))

while a > 0 and b > 0:
    if a > b:
        a %= b
    else:
        b %= a

print("НСД =", a + b)
```